Project Specification:
Intelligent Customer Service Agent
(ReAct + LangGraph)

1 Objective

Develop a natural language-driven Customer Service Agent that can:

- Understand customer queries (orders, complaints, returns)
- Retrieve structured data from MySQL
- Interact with tools dynamically (ReAct)
- Maintain:
  - Short-term memory (session context)
  - Long-term memory (customer profile and history)
- Generate accurate, personalized responses

2 System Overview

2.1 Core Paradigm: ReAct (Reason + Act)

The agent follows:

1. Reason about user intent
2. Select tools dynamically
3. Execute actions (DB/API calls)
4. Update memory
5. Generate response

2.2 LangGraph Workflow

```
User Input
   |
   v
[Planner Node]
   |
   v
[Tool Node(s)]
   |
   v
[Memory Update Node]
   |
   v
[Verifier Node]
   |
   v
Final Response
```

3 LangGraph Node Design

3.1 Planner Node (LLM Reasoning)

- Extract intent (refund, tracking, complaint)
- Extract entities (order_id, product, date)
- Generate execution plan

Example:
Query: Where is my order 123?
Plan:

- Call OrderLookupTool
- Retrieve status
- Generate response

3.2 Tool Execution Node

- Executes MySQL queries
- Calls APIs
- Performs business logic

3.3 Memory Node

3.3.1 Short-Term Memory (STM)

- Stored via LangGraph checkpointer
- Contains:

  - Recent messages
  - Tool outputs

3.3.2 Long-Term Memory (LTM)

- Stored in MySQL
- Optional vector database
- Contains:
  - Customer preferences
  - Interaction history
  - Issue patterns

3.4 Verifier Node

- Ensures correctness of responses
- Prevents hallucinations
- Enforces policy compliance

4 Tool Design

4.1 OrderLookupTool

Function: Retrieve order details

```
SELECT * FROM orders WHERE order_id = ?;
```

4.2 CustomerProfileTool

```
SELECT * FROM customers WHERE customer_id = ?;
```

4.3 RefundTool

```
UPDATE orders SET status='refund_requested' WHERE order_id = ?;
```

4.4 ComplaintLoggerTool

```
INSERT INTO complaints (...) VALUES (...);
```

- Suggest products based on customer history

5 MySQL Database Design

5.1 Customers Table

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.2 Orders Table

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_name TEXT,
    status VARCHAR(50),
    order_date TIMESTAMP,
    delivery_date TIMESTAMP
);
```

5.3 Complaints Table

```
CREATE TABLE complaints (
    complaint_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_id INT,
    issue TEXT,
    status VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.4 Customer Memory Table

```
CREATE TABLE customer_memory (
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    key TEXT,
    value TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

6 Memory Design

6.1 Short-Term Memory

```
MessagesState = {
    "messages": [...]
}
```

- Tracks conversation history
- Maintains context across turns

6.2 Long-Term Memory

- Stored in MySQL
- Includes:
  - Preferences (e.g., refund preference)
  - Past complaints

6.3 Example

User: Last time my delivery was late again
Agent:

- Query long-term memory
- Detect repeated issue
- Adjust response and priority

7 Example ReAct Execution

Query: I want a refund for order 5678

7.1 Steps

1. Planner Node identifies intent and entity
2. Tool Node calls:
   - OrderLookupTool
   - RefundTool
3. Memory updated
4. Verifier confirms correctness
5. Response generated

Final Response:
Your refund request for order 5678 has been successfully initiated.

8 Key Features

8.1 Intelligent Behavior

- Natural language understanding
- Multi-step reasoning (ReAct)

8.2 Tool Integration

- MySQL queries
- Business logic execution

8.3 Memory Awareness

- Short-term session memory
- Long-term personalization

8.4 Robustness

- Verifier reduces hallucination
- Structured workflow ensures reliability

9 Minimal Test Score List (One Query per Function)

This section provides a concise scoring checklist where each query tests a single function. It is suitable for quick grading or unit testing.

9.1 Test Cases

#	Function	Test Query	Expected Behavior
1	Intent Parsing	Where is my order 12345?	Extract intent = tracking, order_id
2	OrderLookupTool	Check status of order 1001	Query MySQL orders table
3	CustomerProfileTool	Show my profile	Retrieve customer info
4	RefundTool	Refund order 5678	Update order status
5	ComplaintLoggerTool	I want to complain about order 2222	Insert complaint record
6	Multi-step Reasoning	Refund order 7890 if delivered	Check then perform refund
7	Short-Term Memory (STM)	Cancel it (after prior order query)	Use previous order_id
8	Long-Term Memory (Read)	What issues have I had before?	Retrieve from memory.md
9	Long-Term Memory (Write)	Remember I prefer refunds	Update YAML/Markdown
10	Personalization	My order is late again	Detect repeated issue
11	Verifier	Refund order 0000	Reject invalid order

10 Conclusion

This system evolves from a simple RAG-based assistant into a full transactional AI system by integrating:

- Structured workflows (LangGraph)
- ReAct reasoning
- Real database interaction
- Persistent memory